Class 6: R functions

Julianne Lee (A17979401)

Table of contents

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

A first function

Here we will create a function to add some numbers. Let's call it `add()`

Input arguments can be either **required** or **optional**. The later have fall-back default values that will be used if the user does not specify them.

> **Q1.** Write a first simple function `add()` to add numbers together. Make sure to include comments in your function explaining what it does and why you do things the way you do.

a. Your first version of the function should add two input numbers together. For example, add(4,

7) should return 11.

```
add <- function(x, y=0) {
  x + y
}
```

Can we use our new function:

```
add(10, 100)
```

```
[1] 110
```

```
add (10)
```

```
[1] 10
```

b. For you second version, adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add(c(4,7,10)).

```
add <- function(x,y=0) {
  sum(x,y)
}
```

```
add(c(1,2,3,4))
```

```
[1] 10
```

```
add(4,7)
```

```
[1] 11
```

c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) should return 2. Hint: explore the … (dots) argument or the base R function sum()

```
add <- function(...) {
  sum(...)
}
```

```
add(1,2,3,-4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the `return()` function call.

```
add <- function(x, y=0, z=0) {
  return(sum(x,y,z))
  cat("Is it break time yet?\n")
}

add(10,100)
```

```
[1] 110
```

A generate_dna() function

A useful function here is the "base R" `sample()` function:

```
sample(1:5, size=3)
```

```
[1] 5 1 4
```

```
sample(1:5, size=7, replace=TRUE)
```

```
[1] 2 2 3 4 1 3 4
```

We can use this to make a random nucleotide sequenc if we draw from "A", "C", "G" and "T" ...

```
sample(c("A","C","G","T"), size=10, replace=TRUE)
```

```
 [1] "A" "A" "A" "C" "G" "A" "G" "A" "A" "A"
```

Q2. Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user.

a. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return "A", "T", "T", "G", "A", "C".

```
generate_dna <- function(len) {
  sample(c("A","C","G","T"), size=len, replace=TRUE)
}
```

```
generate_dna(6)
```

```
[1] "G" "C" "C" "G" "T" "A"
```

b. Your second version should *optionally* be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a singe vector of multiple letters). For example "AAGGCTG".

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`

```
generate_dna <- function(len, single=TRUE) {
  dna<- sample(c("A","C","G","T"), size=len, replace=TRUE)
  if(single) {
    paste(dna, collapse="")
  } else {
    dna
  }
}
```

```
generate_dna(10)
```

```
[1] "AAGCCTGCTG"
```

```
generate_dna(10,F)
```

```
 [1] "A" "T" "A" "A" "T" "A" "C" "A" "T" "G"
```

c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example:

```
>len9
CGAAGGCTG
```

```
generate_dna <- function(len) {
  dna<- sample(c("A","C","G","T"), size=len, replace=TRUE)
  fasta <- paste(dna, collapse="")
  cat(">len", len, sep="")
  cat("\n")
  cat(fasta)
}
```

```
generate_dna(9)
```

```
>len9
ACTGCTAGC
```

Write a `generate_protein()` function

> **Q3.** Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return `"WQRTAG"`.

```
generate_protein <- function(len) {
  protein <- sample(c("M","A","P","G","I","L","V","C","N","Q","T","S","F","Y","W","K","R","Y
                      size=len, replace=TRUE)
  paste(protein, collapse="")
}
```

```
generate_protein(6)
```

```
[1] "IRLVYS"
```

Generate random protein sequences of length 6 to 13

> **Q4.** Adapt and use your generate_protein() function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function for() or sapply() so that you do not have to call generate_protein() eight times by hand.

```r
for (i in 6:13) {
  cat(">id.",i, "\n", sep="")
  cat(generate_protein(i),"\n")
  }
```

```
>id.6
TYWWMY
>id.7
DFYESYL
>id.8
ENQAARTQ
>id.9
DATEGYLVQ
>id.10
PKYPMGWIVQ
>id.11
QTMMYTMYMDG
>id.12
KPYAVNEICDWY
>id.13
VDCYEQYKCLQWV
```

Are Our peptides "unique in nature"?

Q5. Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at https://blast.ncbi.nlm.nih.gov/. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
6	100.00%	100%	N
7	100.00%	100%	N
8	100.00%	100%	N
9	88.89%	100%	Y
10	100.00%	80%	Y
11	81.82%	91%	Y
12	80.00%	83%	Y
13	75.00%	92%	Y

Then answer the following in 1–3 sentences each:

a. At which sequence length do your randomly generated peptides start to look "unique in nature" (i.e. no 100% coverage + 100% identity hit)?

   From 9

b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20 for a peptide of length) and to the size of the known protein universe

   Very short random peptides are almost always found in nr as the number of possible combination (20) is relatively small to the longer peptides. While the size of known protein universe is 214M, 6 amino acids only have 64M space while 10 amino acids have 10.3T space.

Connecting your findings to immunology

Q6. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

I think minimum length will be around 9 amino acids as my sequences over 9 started to lose uniqueness. Short peptides are poor design choice as they are not unique proteins. This will make immune system difficult to distinguish the pathogens and body proteins, leading to autoimmune diseases.